Contents

PLAN — Per-element heterogeneous-material dispatch for volume, BC, and fault faces (Phase H Stage 2)

Date: 2026-05-20 Branch: `feature/heterogeneous_riemann_solver` Goal: Let `seas_spatial_dyn_driver` run TPV31 (and any future benchmark with a depth-varying / `depth_profile_1d` material) end-to-end through the heterogeneous `WaveOperator(MaterialField, BoundaryConfig)` ctor, by wiring per-element material into the three remaining scalar-`flux_` code paths and relaxing the `R-002` guard.

This is the "Phase H Stage 2 follow-up" named in the guard at `dynamic/wave_operator.inl:638` and the "OUT OF SCOPE for Phase R" note at `:613`.

---

1. Motivation / what is blocked today

TPV31 uses a discontinuous 1D velocity structure (`[material] kind = "depth_profile_1d"` → `MaterialField::Mode::Coefficient`). That is mutually exclusive with the scalar `WaveOperator` ctor, so the benchmark must use the heterogeneous `(MaterialField, BoundaryConfig)` ctor. But that ctor hard-aborts at `wave_operator.inl:627-639` (R-002 guard) whenever `Mode::Coefficient` is combined with any real BC (`absorbing_attrs` / `natural_attrs` / `dirichlet_attrs` non-empty, or `fault_attr > 0`). TPV31 has `absorbing=[103,104]`, `natural=[102]`, `fault=101`, so it aborts at construction.

Reproduced 2026-05-20 (after the mesh fix below) via:

```
./seas_spatial_dyn_driver --config tpv31/configs/tpv31.toml \
    --mesh /tmp/tpv31_coarse.msh --dry-run
# ... mesh + material parse OK (5 layers printed) ...
# Verification failed: (material.mode == Mode::Constant || !has_real_bc)
#   --> wave_operator.inl:639  (R-002 guard)
```

Mesh prerequisite (already fixed, separate from this plan): `tpv31/mesh/tpv31_50m.geo` had two bugs that prevented gmsh from producing a volume mesh at all; both are fixed and verified (coarse `-clscale 10` build completes, gmsh exit 0): 1. Added `Curve{23} In Surface{2};` — the surface-reaching fault's top edge lies in the free-surface plane and must be embedded so the 2D free-surface mesh conforms to it (was: segment-facet

intersection / "No elements in volume 1"). Mirrors `Line{101} In Surface{1}` in `tpv205/mesh/tpv2053d_200m.geo`. 2. Removed `Mesh.OptimizeNetgen = 1;` — the standard gmsh optimizer reaches "No ill-shaped tets"; the extra Netgen pass aborts (SIGABRT, illegal tets) on this geometry. TPV205/102/104 do not set it.

The mesh fix alone is not sufficient: the dry-run then hits the R-002 guard, which is what this plan removes the need for.

---

2. Root cause — full inventory of scalar-`flux_` sites

The heterogeneous ctor (`wave_operator.inl:582`) already builds the per-element machinery: - `owned_flux_pool_` (`GodunovFluxPool`) — `At(e)` returns a `GodunovFlux` built from element `e`'s `(λ,μ,ρ)`; carries the correct per-element Jacobians, star matrices, impedances, and all BC methods (`AbsorbingTotal`, `FreeSurfaceTotal`, `FreeSurfaceGodunovTotal`, `Interior`, `Central`). (`BuildGodunovFluxPool_`, `:670–729`) - `per_elem_lmr_` — per-element `(λ,μ,ρ)`. - `per_face_bimaterial_flux_` — per-(face,side) precomputed interior flux matrices. - `shared_face_neighbour_material_` — cross-rank neighbour `(λ,μ,ρ)`.

Interior non-fault faces are ALREADY heterogeneous (Phase R): the `else if (owned_flux_pool_)` branch at `:3467` (RK4 local) and `:4056` (ADER local) routes through `BimaterialFlux::ApplyPerFaceFlux`. Shared interior faces likewise. Those are DONE and out of scope here.

The remaining scalar-`flux_` sites — the ones that produce wrong physics on a depth-varying mesh — fall in three groups:

Group A — Volume term (bulk wave propagation; MOST critical for TPV31's 1D velocity profile)

Site	File:line	Uses
A1	`wave_operator.inl:1627–1629` (`ComputeVolumeRHS`, RK4)	scalar members `Ax_/Ay_/Az_` (built once at ctor `:55–57` via `flux_.BuildJacobian`). Genuine per-element `for e` loop — fix is an in-loop `Flux-ForElem_(e).GetReferenceStarMatrix(0/1`
A2	`wave_operator.inl:1868` (`ComputeADERTimeIntegrated`)	one global `A_d = flux_.GetReferenceStarMatrix(d)` applied to ALL DOFs via `ApplyJacobianPerDOF` (`:1779`). NOT a per-element loop.
A3	`wave_operator.inl:1987` (`ComputeADERSubStepStates`)	same global `ApplyJacobianPerDOF(A_d, ...)`. NOT a per-element loop.

A1 is a genuine per-element loop — the fix is a cheap in-loop `At(e).GetReferenceStarMatrix(d)` fetch (the star matrices are cached const refs precomputed in each `GodunovFlux` ctor — no recompute).

A2 and A3 are NOT per-element loops. `ComputeADERTimeIntegrated` and `ComputeADERSubStepStates` operate on GLOBAL vectors and apply ONE 9×9 Jacobian to the entire field via the free function `ApplyJacobianPerDOF(A_d, dQ_dxd, D_next, ndof_total_, -1.0)` (`wave_operator.inl:1868`, `:1987`). `ApplyJacobianPerDOF` (`:1779–1810`) loops `for (int i = 0; i < ndof_total; i++)` applying the SAME `A` to every DOF — there is no element index in scope. Making A2/A3 heterogeneous therefore requires restructuring `ApplyJacobianPerDOF` (or its callers): a

per-element variant that loops elements and applies `FluxForElem_(e).GetReferenceStarMatrix(d)` to element `e`'s `ndof_per_el_` DOFs (component stride `ndof_total_`). This is the largest single piece of Phase 1 — NOT a "structurally local" fetch. TPV31 runs ADER (`ader_order = 2` in `tpv31.toml`; `--ader-order 3` in the p2/O3 sbatch), so the CK predictor is squarely on the critical path.

> NOTE: `GodunovFlux::GetReferenceStarMatrix`'s own doc (`godunov_flux.hpp:208-212`) flags this exact gap: "Future extension to heterogeneous material … will need either per-element star matrices or a reference-frame CK path." The per-element pool supplies the former, but it must be threaded through a per-element `ApplyJacobianPerDOF` (A2/A3), not just fetched in an existing loop.

Group B — Boundary face flux (absorbing / free-surface / natural)

Site	File:line	Uses
B1	`wave_operator.inl:3041, 3054, 3059, 3095` (`ComputeFaceFluxRHS`, RK4 local)	`flux_.AbsorbingTotal / FreeSurfaceGodunovTotal / FreeSurfaceTotal`
B2	`wave_operator.inl:5007, 5012, 5017, 5027` (`ComputeADERFaceFluxRHS`, ADER local)	same set

BC faces are one-sided (`e2 < 0`); the local element is `e1`. The fix is `flux_ → owned_flux_pool_->At(e1)` (the pool's `GodunovFlux` carries the BC methods verbatim). BC faces never appear in the shared RHS functions, so no shared-side BC change is needed.

Group C — Fault bulk-side flux (the **`A_n · Q_imp`** contribution)

The friction solve and imposed-state construction are already heterogeneous-ready: `FaultFaceFlux::Evaluate / EvaluateADER*` read impedances/η entirely from per-DOF `DOFData` (`Zp_plus/Zp_minus/Zs_plus/Zs_minus/eta_p/eta_s`, `fault_face_flux.cpp:49-67, 184, 234-271`), and the spatial driver already populates those per-DOF from the material field via `material.EvalAt(...)` in `seed_static_dof_fields` (`spatial_setup.hpp:71-82, 117-128`) — depth-varying when the material is `Mode::Coefficient`. Rotation matrices (`GodunovFlux::BuildRotation*`) are static/material-free.

What is not heterogeneous is the bulk-side flux that gets assembled into the DG RHS: each side applies `A_n · Q_imp` via the scalar `flux_.Interior(can_n, Q_imp, Q_imp, F_h)`: | Site | File:line | Path | |---|---|---| | C1 | `wave_operator.inl:3335-3338` (`ComputeFaceFluxRHS`) | RK4 local fault | | C2 | `wave_operator.inl:4013` (`ComputeSharedFaceFluxRHS`) | RK4 shared fault | | C3 | `wave_operator.inl:4603-4606` and the per-QP-batched variant `:4921-4924` (both inside `ComputeADERFaceFluxRHS`, `:4140-5183`) | ADER local fault — TWO blocks | | C4 | `wave_operator.inl:5568` (`ComputeADERSharedFaceFluxRHS`, `:5183+`) | ADER shared fault |

> NOTE: `ComputeADERFaceFluxRHS` (ADER local) has TWO fault dispatch blocks (`:4603-4606` and the per-QP-batched `:4921-4924`). Both must be edited. `:4921-4924` is NOT in `ComputeADERSharedFace-FluxRHS` despite the per-QP naming — that function starts at `:5183`.

For TPV31 the fault is vertical with a depth-only material, so both sides at a given fault QP have identical material (the existing `Zp_plus ≈ Zp_minus` assertions at `fault_face_flux.cpp:332-340` etc. hold). Each side's bulk flux should use that side's fault-adjacent element via `At(e1)` / `At(e2)` for local interior fault faces; on shared fault faces each rank has only ONE local element, so use `At(local_elem)` (do NOT call `At(e2)` on a shared face — `e2` is off-rank). True bi-material across the fault is out of scope (see §7).

Group D — The guard itself

`wave_operator.inl:621-640` (R-002). Once A–C are wired, relax it so `Mode::Coefficient` + real BC is allowed.

Already heterogeneous — do NOT change

`ComputeMaxDt` (`wave_operator.inl:5921`) already branches on `!per_elem_h_.empty()` (`:5974`) and computes the stable timestep per-element from `per_elem_lmr_`/`per_elem_h_` (`:5985–6001`, real per-element `c_p`). The `flux_.GetCp()` at `:6004` is the scalar-ctor fallback and MUST stay (the placeholder material is never reached because the heterogeneous ctor populates `per_elem_h_`). A blanket `flux_ → At(e)` sweep must NOT touch `:6004`.

---

3. Design principle (safety contract)

Single dispatch rule, applied at every A/B/C site: > use the per-element flux when the pool exists, else the scalar member.

Concretely, add one private helper:

```
// wave_operator.hpp (private)
const GodunovFlux& FluxForElem_(int e) const
{
   return owned_flux_pool_ ? owned_flux_pool_->At(e) : flux_;
}
```

and (for the volume term) per-direction Jacobian access via `FluxForElem_(e).GetReferenceStarMatrix(d)` for `d = 0,1,2`. This is the ONLY per-direction accessor: `GodunovFlux` exposes `GetAx()` (`godunov_flux.hpp:176`) but not `GetAy()`/`GetAz()`. A1 currently reads the cached members `Ax_/Ay_/Az_` (numerically equal to `GetReferenceStarMatrix(0/1/2)`); switch it to the element's `GetReferenceStarMatrix(d)`.

Invariants this preserves (the regression contract): 1. Scalar-ctor drivers (TPV205/102/104, BP5) are byte-untouched. They never set `owned_flux_pool_` (it is `nullptr`), so `FluxForElem_` returns `flux_` and every code path is bit-for-bit the pre-change path. No `git diff` to their output. 2. **`Mode::Constant`** stays byte-identical — but ONLY IF the pool's cached flux is built from EXACT material. `GodunovFluxPool::Build` currently builds the cached `GodunovFlux` from 6-sig-fig-ROUNDED `(λ,μ,ρ)` (`godunov_flux_pool.cpp:103–106`), so `At(e)` differs from the exact-material `flux_` at ~1e-6 for any constant with >6 significant figures. Today this is masked because the volume term still uses the exact `Ax_`; Phase 1 (A1) removes that mask by routing the volume term through `At(e)`. Therefore Phase 1 must first build the pool from exact values (round only the dedup key — see Phase 1 prerequisite and R-002). The existing parity test passes today only because it uses clean constants (`32.0e9 / 2670`); the extended test (§6) must use BOTH a clean and a >6-sig-fig constant so the gate actually catches rounding regressions. 3. Interior-face bimaterial path is untouched (already correct).

---

4. Phased implementation

Each phase is independently buildable and gated by the Mode::Constant byte-parity test before moving on. Recommended order is by physical blast radius: volume first (it governs the bulk 1D-velocity physics that is the whole point of TPV31), then BC, then fault, then the guard.

Phase 1 — Volume / ADER-CK per-element Jacobians (Group A)

Prerequisite (do FIRST — see R-002): make the flux pool build its cached `GodunovFlux` from EXACT material so the Mode::Constant byte-parity gate below can hold. Change `dynamic/godunov_flux_pool.cpp:106` to `std::make_unique<GodunovFlux>(lam, mu, rho)` (build from the exact values; keep `make_key`'s `round_sig` for the dedup KEY only). Re-run `seas_test_phaseh_wave_operator_constant_parity` — its only behavior change is at the LSB for >6-sig-fig heterogeneous input, which has no scalar reference.

Phase 1a — RK4 volume (A1; simple, genuine per-element loop)

- Files: `dynamic/wave_operator.inl` (`ComputeVolumeRHS :1584`), `dynamic/wave_operator.hpp` (add `FluxForElem_`).
- Change: inside the existing `for (int e ...)` loop, replace the scalar `Ax_/Ay_/Az_` (`:1627–1629`) with `FluxForElem_(e).GetReferenceStarMatrix(0/1/2)`.

Phase 1b — ADER CK recursion (A2/A3; requires restructuring)

- Files: `dynamic/wave_operator.inl` (`ApplyJacobianPerDOF :1779`, `ComputeADERTimeIntegrated :1816/:1868`, `ComputeADERSubStepStates :1913/:1987`).
- Change: these are NOT per-element loops (one global Jacobian over all `ndof_total_` DOFs). Add a per-element variant of `ApplyJacobianPerDOF` that loops elements `e`, fetches `FluxForElem_(e).GetReferenceStarMatrix(d)`, and applies it ONLY to element `e`'s `ndof_per_el_` DOFs (offset `e*ndof_per_el_`, component stride `ndof_total_`). Route both CK call sites (`:1868`, `:1987`) through it when `owned_flux_pool_` is set; keep the global scalar path when it is null.
- Acceptance (both 1a + 1b):
  - Mode::Constant MaterialField ctor: byte-identical volume RHS + ADER predictor vs scalar ctor (extend parity test; use BOTH a clean and a >6-sig-fig constant — see R-002).
  - 1D layered-slab heterogeneous correctness test (§6 H-1): a plane P-wave crossing a velocity discontinuity produces the analytic reflection/transmission amplitude (within DG discretization error), which it CANNOT with a single scalar Jacobian.

Phase 2 — Boundary-face per-element flux (Group B)

- Files: `dynamic/wave_operator.inl` (`ComputeFaceFluxRHS` BC block `:3026–3097`, `ComputeADERFaceFluxRHS` BC block `:4995–5030`).
- Change: `flux_.{AbsorbingTotal,FreeSurfaceTotal, FreeSurfaceGodunovTotal}` → `FluxForElem_(e1).{...}` (BC faces are one-sided; `e1` is the owning element, already in scope as `ftr->Elem1No`).
- Acceptance:
  - Mode::Constant byte-identical (parity test extension).
  - Heterogeneous absorbing test (§6 H-2): an outgoing wave hits an absorbing wall in a layered medium with ~no reflection at the material-correct impedance; reflection coefficient ≪ the scalar-placeholder case.

Phase 3 — Fault bulk-side per-element flux (Group C)

- Files: `dynamic/wave_operator.inl` fault branches at the four sites in §2-C.
- Change: the two `flux_.Interior(can_n, Q_imp_*, Q_imp_*, F_h_*)` (plus/minus side) → use the per-side element's flux: `FluxForElem_(elem_plus).Interior(...)` / `FluxForElem_(elem_minus).Interior(...)`. Determine `elem_plus` / `elem_minus` from the existing `*_elem1_on_plus_` bookkeeping (`interior_fault_elem1_on_plus_`, `shared_fault_elem1_on_plus_`) — for local interior faults the two sides are `e1`/`e2`; for shared faults each rank uses its single local element.
- Keep: the `Zp_plus ≈ Zp_minus` assertions (TPV31 is homogeneous across the fault). Do not attempt bi-material-across-fault here.
- Acceptance:
  - Mode::Constant byte-identical.
  - TPV31 fault QP impedances vary with depth (already true via `seed_static_dof_fields`); add a unit check that the bulk-side flux at a shallow vs deep fault QP differs (scalar would make them equal).

Phase 4 — Relax the guard + TPV31 end-to-end

- Files: `dynamic/wave_operator.inl:621–640` (R-002 guard).
- Change: remove/relax the `has_real_bc` rejection for `Mode::Coefficient`. Replace with a NARROWER guard that still rejects the genuinely-unsupported case (e.g. true bi-material across the fault, if cheaply

detectable) and/or a one-time informational banner. Keep the `MixedFluxMode::None` requirement (`:652`) unchanged.
- Acceptance:
  - `seas_spatial_dyn_driver --config tpv31/configs/tpv31.toml --mesh <coarse>.msh --dry-run` reaches `[spatial_dyn] --dry-run: construction complete` (exit 0).
  - Short real run (coarse mesh, few hundred steps) nucleates: hypocenter `|V_strike|` rises off zero; no NaN; bimaterial banner present.
  - The two `tpv31_spatial` sbatch jobs are unblocked.

---

5. Interfaces touched (summary)

- New: `WaveOperator<MeshType>::FluxForElem_(int e) const` (private, header-inline). No public API change.
- Modified (bodies only, signatures unchanged): `ComputeVolumeRHS`, `ComputeADERTimeIntegrated`, `ComputeADERSubStepStates`, `ComputeFaceFluxRHS`, `ComputeADERFaceFluxRHS`, `ComputeSharedFaceFluxRHS`, `ComputeADERSharedFaceFluxRHS`, and the R-002 guard block in the heterogeneous ctor.
- Unchanged: `GodunovFlux`, `GodunovFluxPool`, `BimaterialFlux`, `FaultFaceFlux`, all scalar-ctor call sites, the interior-face bimaterial path, `spatial_setup.hpp` (per-DOF impedances already correct), every TPV/BP5 driver.

---

6. Test strategy

Regression / parity (must pass after every phase): - `seas_test_phaseh_wave_operator_constant_parity` (np=1 + np=4) — EXTEND to construct a `Mode::Constant` MaterialField operator WITH absorbing + free-surface + fault BCs and assert byte-identical `Mult` and `AdvanceADER` output vs the scalar ctor. Today this test only exercises the path that the guard allowed (no real BC); the guard relaxation makes the BC/fault paths reachable, so the parity test must cover them. - Full `make test` green; TPV205/102/104 byte-parity drivers unchanged (they use the scalar ctor → structurally untouched, but run the existing parity smoke to be safe).

Heterogeneous correctness (new — proves the fix does something real): - H-1 (volume): 1-D layered slab, two materials with a velocity jump, plane P-wave at normal incidence. Compare numerical reflection/transmission coefficients against the analytic impedance-contrast values `R = (Z2−Z1)/(Z2+Z1)`. A scalar Jacobian cannot reproduce a non-zero R at an internal material jump; the per-element volume term must. - H-2 (BC): same slab, outgoing wave into an absorbing wall sitting in the lower-velocity layer; reflection ≪ placeholder-material case. - H-3 (fault): reuse the existing `test_phaser_dispatch_smoke` bi-material fixture pattern; assert the fault bulk-side `A_n·Q_imp` differs between a shallow and a deep fault QP (different `(λ,μ,ρ)`), whereas the scalar path makes them equal.

Integration: - TPV31 dry-run (Phase 4 acceptance) on the coarse mesh (`gmsh −clscale 10`), then on the full 50 m mesh once available. - TPV31 short run: confirm spontaneous nucleation at the hypocenter (total hypo shear 34.95·μ/μ0 > yield 34.80·μ/μ0, per the spec-exact config), `V_max` rises then the front propagates; no NaN tripwire.

---

7. Out of scope (explicitly)

- True bi-material across the fault (different material on the + and − sides of the fault plane). TPV31's fault is vertical in a depth-only medium → both sides share material at every QP. The `Zp_plus ≈ Zp_minus` assertions stay; lifting them is a separate effort (would need per-side DOFData impedances + a bi-material fault Riemann solver).
- **`MaterialField::Mode::GridFunction`** (no centroid eval for the pool; same restriction as today, `wave_operator.inl:594–601`).

- MixedFlux + heterogeneous combination (already rejected at `:652`; unchanged).
- Performance tuning beyond the existing pool dedup; the per-element fetch is a cached-reference lookup, so the hot-loop cost is a pointer indirection, not a recompute. (Revisit only if profiling on the full 50 m mesh shows it matters.)

---

8. Risks & mitigations

- Byte-parity drift on Mode::Constant — CONFIRMED, not hypothetical. `GodunovFluxPool::Build` builds the cached flux from the rounded `(λ,μ,ρ)` (`godunov_flux_pool.cpp:103–106`), so `At(e)` ≠ exact `flux_` for >6-sig-fig constants. The existing parity test passes ONLY because it uses clean constants (`32.0e9 / 2670`, `test_phaseh_wave_operator_constant_parity.cpp:88–90`). Mandatory mitigation (Phase 1 prerequisite): build the cached flux from EXACT values (`godunov_flux_pool.cpp:106` → `make_unique<GodunovFlux>(lam, mu, rho)`), rounding only the dedup KEY; and have the extended parity test use BOTH a clean and a >6-sig-fig constant so the gate catches regressions.
- Fault side/element identification on shared faces. Use the existing `*_elem1_on_plus_` swap flags rather than re-deriving orientation; add an assertion that the chosen element index is local and in range.
- Volume term is the highest-value, highest-risk change (it governs every interior DOF). Do Phase 1 first and gate hard on H-1 before touching BC/fault.
- Scope creep toward bi-material-across-fault. Keep the `Zp_plus ≈ Zp_minus` assertions in; they are the tripwire that keeps this change honest for TPV31 and flags any future config that violates the assumption.

---

9. References

- Guard / known limitation: `dynamic/wave_operator.inl:603–640`.
- Per-element pool: `dynamic/godunov_flux_pool.hpp`, `BuildGodunovFluxPool_ wave_operator.inl:670–729`.
- Interior bimaterial path (the template to mirror): `wave_operator.inl:826–1130` (`BuildPerFaceBimaterialFluxMatrices_`), dispatch at `:3467`, `:4056`.
- Fault flux per-DOF impedances: `dynamic/fault_face_flux.cpp:49–271`; per-DOF seeding `dynamic/spatial_setup.hpp:4142`.
- Phase R plan (interior faces, prior art): `safs/project_7.0_alternative/document/PLAN_phase_R_exact_bimaterial_riema`
- TPV31 spec-exact config + mesh fix: `tpv31/configs/tpv31.toml`, `tpv31/mesh/tpv31_50m.geo`.